FH JOANNEUM - University of Applied Sciences

Dynamic Resource Allocation and Scaling of Stateful Workloads in Kubernetes

A Comparative Study of Reactive and Predictive Scaling Approaches for Redis Clusters

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

FH | JOANNEUM
University of Applied Sciences

Ehrenwörtliche Erklärung

Hier steht die ehrenwörtliche Erklärung.

Table of Contents

Ehrenwörtliche Erklärung i
Table of Contents ii
List of Figures iii
List of Tables iv
List of Abbreviations v
Kurzfassung vi
Abstract vii
1 Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
2 Theoretical Foundations 2
2.1 Kubernetes Concepts Overview 2
2.1.1 Use Cases of Kubernetes 2
2.2 Autoscaling in Kubernetes 2
2.3 Stateful vs. Stateless Workloads 2
2.3.1 Problems with Stateful Workloads in Kubernetes 2
2.4 Metrics and Observability in Kubernetes 2
3 Related Works into Kubernetes Autoscaling with Stateful Workloads . . . 3
3.1 Existing Research into Stateful Kubernetes Autoscaling 3
3.2 Industry Implementations and Best Practices 3
3.3 Research Gaps in Current Research and Opportunities for Improvement 3
4 Methodology and Experimental Setup 4
4.1 Experimental Setup 4
4.1.1 Virtualisation Environment 4
4.1.2 Routing 4
4.1.3 Redis Cluster Configuration 4
4.1.4 FastAPI Application Design 4
4.2 Autoscaling Configuration 4
4.3 Load Simulation 4
4.4 Evaluation Metrics and Data Collection 4
5 Results and Analysis 6
5.1 Compute and Memory Saturation 6
5.2 Network and Data Pipeline 6
5.3 Application Throughput and Reliability 6
5.4 Final Comparison between Reactive Scaling and Predictive Scaling . . 6
6 Conclusion 7
6.1 Summary of Findings 7
6.2 Discussion 7
Appendix 8
Glossary 9
List of References 10
Attachments 11

List of Figures

1 High-level architecture overview of the experimental setup. 5

List of Tables

List of Abbreviations

API Application Programming Interface

CRD Custom Resource Definition

HPA Horizontal Pod Autoscaler

JSON JavaScript Object Notation

KEDA Kubernetes Event-Driven Autoscaling

K3s Lightweight Kubernetes distribution by Rancher

K8s Kubernetes

yaml YAML Ain’t Markup Language

Kurzfassung

Die Migration zustandsbehafteter Workloads, wie etwa partitionierter Datenbanken wie Redis, nach Kubernetes verursacht erhebliche architektonische Konflikte. Native Kubernetes-Skalierungsmechanismen, insbesondere der Horizontal Pod Autoscaler (HPA) und der `StatefulSet`-Controller, gehen von der generellen Austauschbarkeit von Rechenressourcen aus. Bei Scale-in-Vorgängen verwenden sie einen topologieblinden Löschmechanismus in umgekehrter Reihenfolge, der notwendige Datenmigrationsprotokolle umgeht und einen katastrophalen Datenverlust riskiert.

Diese Arbeit untersucht diese architektonischen Einschränkungen empirisch anhand eines deterministischen, lastfreien A/B-Tests. Die Studie evaluiert den Scale-in eines Redis-Clusters von vier auf drei Master-Knoten mit 5000 Datensätzen (Keys). Dabei wird die native HPA-Baseline mit dem Opstree Redis Operator, gesteuert durch Kubernetes Event-driven Autoscaling (KEDA), verglichen.

Die empirischen Ergebnisse zeigen, dass ein nativer HPA-Scale-in einen massiven Datenverlust verursacht. Dies führte zum sofortigen Verlust von 25 % des Datensatzes (1250 Keys), 4096 verwaisten Hash-Slots und einer clusterweiten Nichtverfügbarkeit. Im Gegensatz dazu fing der Operator den Skalierungsbefehl erfolgreich ab, um einen präemptiven, topologiebewussten Resharding-Prozess zu orchestrieren. Dies ermöglichte einen reibungslosen Scale-in ohne Datenverlust und mit durchgehender Verfügbarkeit.

Letztendlich belegt diese Untersuchung, dass die Implementierung domänenspezifischer Orchestrierungslogik durch das Operator-Pattern nicht nur eine operative Erleichterung, sondern eine absolute architektonische Notwendigkeit darstellt, um die Datenintegrität bei der Skalierung zustandsbehafteter Workloads in Kubernetes zu gewährleisten.

Abstract

The migration of stateful workloads, such as sharded databases like Redis, into Kubernetes introduces significant architectural friction. Native Kubernetes scaling primitives, specifically the Horizontal Pod Autoscaler (HPA) and the `StatefulSet` controller, operate on the assumption of compute fungibility. During scale-in operations, they employ a topology-blind, reverse-ordinal pod deletion mechanism that bypasses necessary data migration protocols, risking catastrophic data loss.

This thesis empirically investigates these architectural limitations through a deterministic, quiescent A/B test. The study evaluates a scale-in operation from four to three master nodes on a Redis cluster containing 5000 keys. It compares the native HPA baseline against the Opstree Redis Operator, driven by Kubernetes Event-driven Autoscaling (KEDA).

The empirical findings demonstrate that a native HPA scale-in induces a severe data cliff. This resulted in the immediate loss of 25% of the dataset (1250 keys), 4096 orphaned hash slots, and cluster-wide unavailability. Conversely, the operator successfully intercepted the scaling command to orchestrate a pre-emptive, topology-aware resharding process. This achieved a graceful scale-in with zero data loss and continuous cluster availability.

Ultimately, this research proves that encoding domain-specific orchestration logic via the Operator pattern is not merely an operational convenience, but an absolute architectural necessity for safeguarding data integrity when scaling stateful workloads in Kubernetes.

1 Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

2 Theoretical Foundations

2.1 Kubernetes Concepts Overview

2.1.1 Use Cases of Kubernetes

2.2 Autoscaling in Kubernetes

2.3 Stateful vs. Stateless Workloads

2.3.1 Problems with Stateful Workloads in Kubernetes

2.4 Metrics and Observability in Kubernetes

3 Related Works into Kubernetes Autoscaling with Stateful Workloads

3.1 Existing Research into Stateful Kubernetes Autoscaling

3.2 Industry Implementations and Best Practices

3.3 Research Gaps in Current Research and Opportunities for Improvement

4 Methodology and Experimental Setup

4.1 Experimental Setup

4.1.1 Virtualisation Environment

4.1.2 Routing

4.1.3 Redis Cluster Configuration

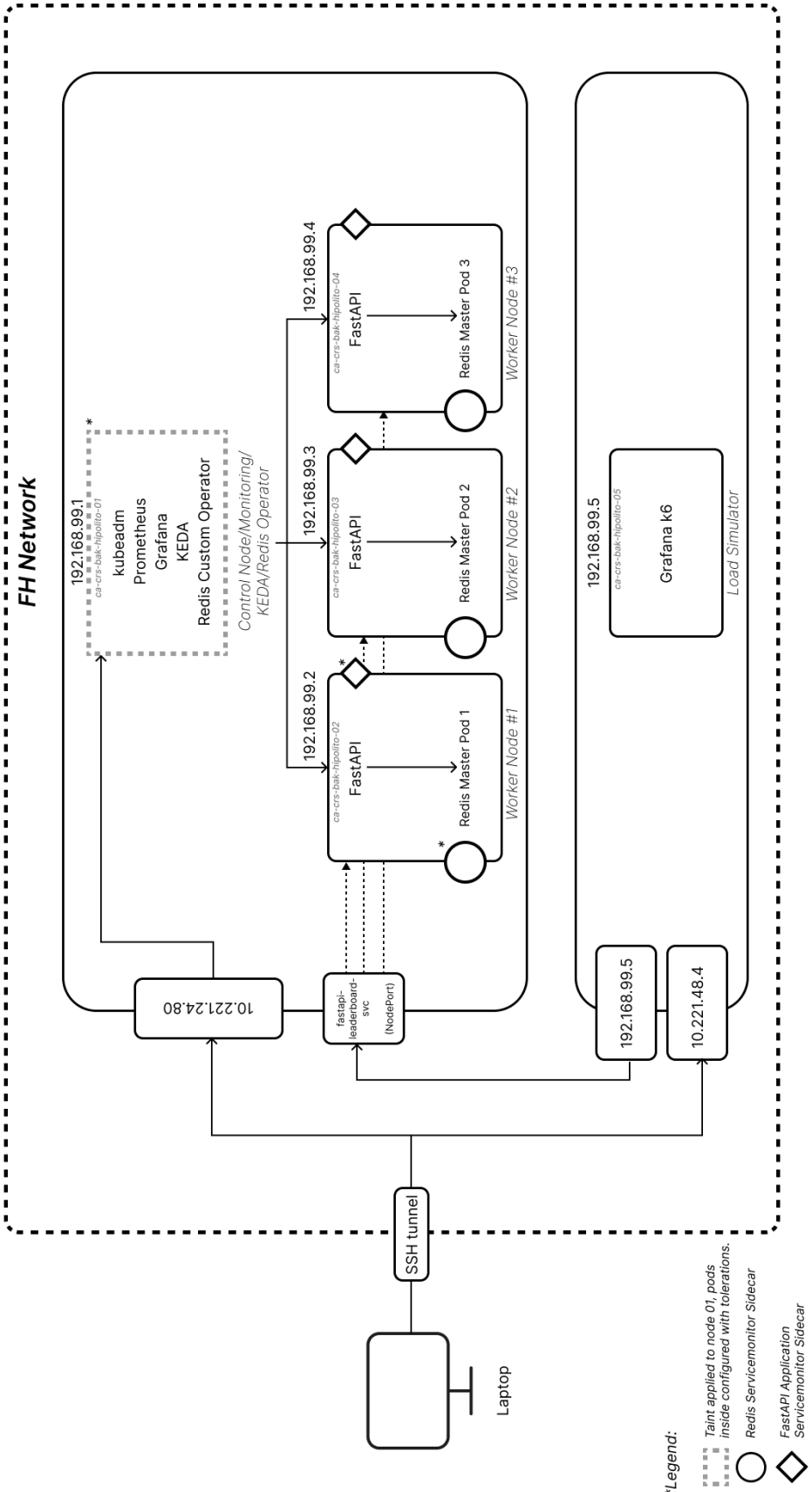
4.1.4 FastAPI Application Design

4.2 Autoscaling Configuration

4.3 Load Simulation

4.4 Evaluation Metrics and Data Collection

Figure 1: High-level architecture overview of the experimental setup.



5 Results and Analysis

5.1 Compute and Memory Saturation

5.2 Network and Data Pipeline

5.3 Application Throughput and Reliability

5.4 Final Comparison between Reactive Scaling and Predictive Scaling

6 Conclusion

6.1 Summary of Findings

6.2 Discussion

Appendix

Glossar

ConfigEpoch A collision-resolution concept in a Redis Cluster used to determine which node holds the authoritative claim over a specific hash slot during split-brain or network partition events.

CRD (Custom Resource Definition) An API extension mechanism in Kubernetes that allows users to define and instantiate custom resources, such as the `RedisCluster` resource utilized by the Opstree operator.

Data Cliff A term utilized in this thesis to describe the sudden, catastrophic loss of data and cluster availability that occurs when a stateful node is terminated without prior data migration.

Ghost Pod A pod provisioned by a generic autoscaler (like HPA) that joins a Kubernetes `StatefulSet` but remains unintegrated into the database topology, meaning it holds zero hash slots and serves no traffic.

Hash Slot The fundamental unit of data partitioning in a Redis Cluster. The total keyspace is divided into exactly $16\,384$ hash slots, which must be distributed across the master nodes to maintain cluster availability.

Horizontal Pod Autoscaler (HPA) A native Kubernetes control loop that automatically updates a workload resource (such as a `StatefulSet`) to match demand based on observed metrics, such as average CPU utilization.

KEDA (Kubernetes Event-driven Autoscaling) A single-purpose, lightweight component added to a Kubernetes cluster to drive autoscaling based on external event metrics, such as Prometheus cache-miss ratios.

Operator Pattern A software extension to Kubernetes that encodes domain-specific operational knowledge into a custom controller to automate the lifecycle and topology management of complex stateful applications.

Quiescent A state of execution where no external read or write traffic is applied to the database. This state is used in the experimental methodology to isolate scaling mechanics from load-induced resharding interruptions.

Resharding The process of live-migrating hash slots, along with the underlying data they contain, from one node to another within an active database cluster.

Stateful Workload An application or database (such as Redis) that persists client data across sessions, meaning its underlying compute pods contain unique data and are not strictly interchangeable.

StatefulSet A native Kubernetes workload API object used to manage stateful applications, providing strict guarantees about the ordering and uniqueness of a set of Pods, including the enforcement of reverse-ordinal deletion during scale-in operations.

List of References

Attachments

The following Attachments are included as a digital supplement to this thesis (as a ZIP archive in the submission system) and reference the complete codebase used for conducting the experiments.

Attachment 1: Project repository (Quellcode)

The complete GitHub repository contains all Kubernetes manifests, Bash automation scripts (`data-safety-experiment.sh`), FastAPI deployment files, and the raw Markdown output logs of the conducted experiments.
URL: https://github.com/gitbudy360/bac-johann-hipolito
Commit-Hash: `be45110ff64e3f3a5527c91c6eba6a5c7270d18a`

Attachment 2: Grafana Dashboards

The JavaScript Object Notation (JSON)-exports of the Grafana dashboards used for visualizing hash slot assignments and key metrics during the scale-in phases. Located in the `/k8s-cluster/grafana/` directory.